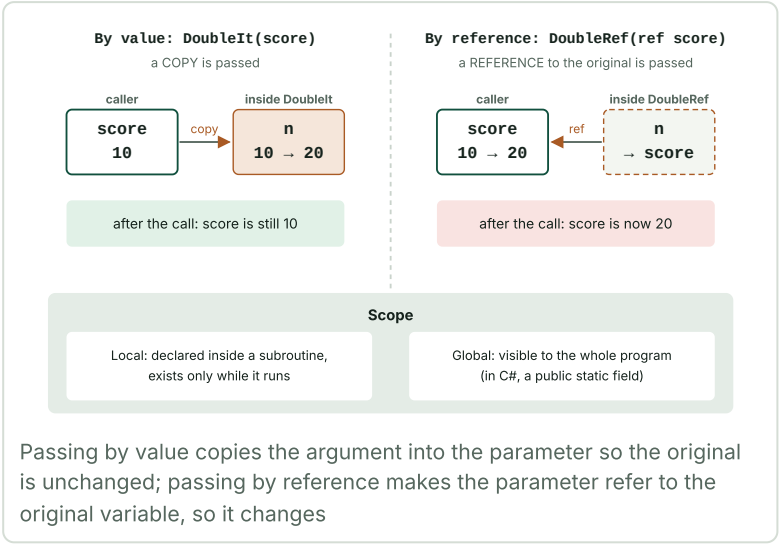
Subroutines: procedures, functions, parameters and scope

AP21 FACT FILE

Specification: students should be able to

- Explain what is meant by a subroutine and describe the difference between a procedure and a function
- Write and call procedures and functions in a high-level language, using parameters, arguments and return values
- Explain passing parameters by value and by reference, and predict the effect of each
- Explain the scope of local and global variables and the advantages of using local variables
- Explain the benefits of modular programming and decomposing a problem into subroutines

Key words: subroutine, procedure, function, method, call, parameter, argument, return value, by value, by reference, scope, local variable, global variable, side effect, modular programming, decomposition, library routine



Passing by value copies the argument into the parameter so the original is unchanged; passing by reference makes the parameter refer to the original variable, so it changes

Overview: building programs from subroutines

As programs grow, writing every instruction in one long block becomes unmanageable. A subroutine is a named, self-contained block of code that performs a specific task and can be called whenever that task is needed. When a subroutine is called, control passes to it; when it finishes, control returns to the statement after the call. There are two kinds. A procedure carries out a task, such as printing a receipt, without returning a value, while a function carries out a task and returns a single value, such as a calculated price, to the point where it was called. Data is passed into a subroutine through parameters, so the same code can work on different values each time. Parameters can be passed by value, where the subroutine receives a copy, or by reference, where it can change the caller's original variable. Each variable also has a scope, the part of the program where it can be used: local variables exist only inside their subroutine, whereas global variables can be accessed anywhere. Splitting a solution into well-designed subroutines is called modular programming, and it makes programs easier to write, test, maintain and share across a team. These ideas build directly on the top-down decomposition used when designing algorithms.

Procedures, functions and methods

In C#, every subroutine is written as a method with a header and a body. The header gives the return type, the name and the parameter list, for example static double AddVat(double price, double rate). A procedure has the return type void, meaning it returns nothing; it is called as a statement on its own, such as ShowMenu();. A function has a return type such as int, double, bool or string and must use a return statement on every path to send back a value of that type. Because it produces a value, a function call is used inside an expression, for example assigned to a variable, printed or tested in a condition: if (IsValidPin(pin)). The return statement also ends the function immediately, so any code after it on that path is not executed. Programs also make constant use of library routines, prewritten and tested subroutines supplied with the language, such as Console.WriteLine, Math.Round and int.Parse. In object-oriented languages, subroutines belonging to a class are called methods, which is why C# uses that term throughout. Choosing between a procedure and a function depends on whether the caller needs a result back: a calculation should normally be a function, while displaying output is usually a procedure.

Parameters, arguments and return values

A parameter is a variable listed in a subroutine's header, together with its data type, that receives a value when the subroutine is called. An argument is the actual value, variable or expression supplied in the call. Arguments are matched to parameters by position, so they must agree in number, order and data type; calling AddVat(40) when two parameters are expected, or passing text where a number is expected, is a compile-time error. Parameters make subroutines general: a single MonthlyCost function can work out the price for any member because the age and the off-peak flag are passed in, rather than being typed into the code. Inside the subroutine, parameters behave like local variables, and each call gets fresh copies of them. A function sends its result back with return, and the value of the whole call expression becomes that result. Designing the interface of a subroutine, meaning its name, parameters and return type, is an important step: if the interface is clear and documented, other programmers can use the subroutine correctly without needing to read the code inside it. Passing data through parameters and return values, rather than through global variables, keeps each subroutine independent and easy to test.

Passing by value and by reference

When an argument is passed by value, the subroutine receives a copy of the value in a new memory location. Any change made to the parameter inside the subroutine affects only the copy, which is discarded when the subroutine ends, so the caller's original variable is protected. This is the default in C# and in most languages, and it is the safer choice whenever the subroutine only needs to read the data. When an argument is passed by reference, the subroutine receives a reference to the original variable, effectively its memory address, so any change to the parameter changes the caller's variable directly. In C# this is requested with the ref keyword in both the header and the call, which makes the intention visible. Passing by reference is useful when a subroutine must update the caller's data, such as swapping two values in a sort, or must give back more than one result; it also avoids copying large amounts of data. The out keyword is a variant in which the subroutine must assign the value, as in int.TryParse. The disadvantage of passing by reference is the risk of side effects: a call can alter variables unexpectedly, making errors harder to find.

Scope: local and global variables

The scope of a variable is the part of a program in which it can be accessed. A local variable is declared inside a subroutine or a block, such as the body of a loop, and can only be used there. It is created when the subroutine is called and destroyed when it ends, so its memory is released and its value does not carry over between calls. Parameters are also local to their subroutine. Because local variables are invisible outside their subroutine, different subroutines can safely reuse common names like count or total without interfering with each other, and each subroutine can be understood, tested and reused on its own. A global variable is declared outside all subroutines and is accessible from anywhere in the program for the whole of its run. C# does not have true global variables, but a public static field of a class behaves in the same way. Globals are convenient for data that genuinely belongs to the whole program, but because any part of the code can change them, an unexpected value can be very hard to trace, modules become dependent on each other and are harder to reuse, and memory is held for the entire run. Good practice is to use local variables and parameters wherever possible.

Modular programming

Modular programming means dividing a program into separate, self-contained modules, such as subroutines, each responsible for one clearly defined task. It follows naturally from top-down design: the main program becomes a short sequence of calls that reads like an outline of the solution, and each call is developed separately. The benefits are significant. Each module can be tested on its own before being combined with others, so errors are easier to locate and fix. A module written once can be called many times and reused in other programs, reducing duplication and development time, and tested modules can be collected into libraries. Maintenance is simpler because a change, such as a new VAT rule, is made in one place. Large projects can be shared across a team, with programmers developing different modules at the same time, provided the interfaces are agreed. There are some costs: planning the modules and their interfaces takes time, and modules must be integrated and tested together as well as separately, because each can work correctly alone yet fail when combined. Overall, though, modular design produces code that is more reliable, readable and maintainable, which is why it is standard practice in industry.

Key terms from the mark schemes

Subroutine
A named, self-contained block of code that performs a specific task and can be called from elsewhere in a program.
Credited: subprogram / method / module

Procedure
A subroutine that performs a task but does not return a value (void in C#).

Function
A subroutine that performs a task and returns a single value to the point where it was called.

Parameter
A variable in a subroutine's header that receives a value (argument) passed in when the subroutine is called.

Argument
The actual value or variable supplied to a subroutine when it is called.
Credited: actual parameter

Passing by value
Passing a copy of an argument to a subroutine so that the original variable cannot be changed by it.
Credited: by val

Passing by reference
Passing the address of (a reference to) a variable so that changes made in the subroutine affect the original variable.
Credited: by ref

Scope
The region of a program in which a variable can be accessed.

Local variable
A variable declared inside a subroutine or block that can only be accessed there and exists only while it runs.

Global variable
A variable that can be accessed from any part of the program and exists for the whole run.

Modular programming
Dividing a program into separate, self-contained modules (subroutines), each performing one clearly defined task.

Library routine
A prewritten, tested subroutine supplied with a language or obtained from elsewhere that can be called in a program.

C C# example

Parameters and arguments

```
static void ShowLine(char symbol, int length)     // parameters
{
    for (int i = 0; i < length; i++) Console.Write(symbol);
    Console.WriteLine();
}

static double AddVat(double price, double rate)
{
    return price + price * rate;
}

ShowLine('=', 20);                                 // arguments
Console.WriteLine("Total: " + AddVat(40, 0.2));    // Total: 48
ShowLine('=', 20);
```

Exam focus

Definition questions reward the key distinguishing point: a function returns a value to the calling code, whereas a procedure does not; a local variable can only be accessed within the subroutine or block in which it is declared. When explaining by value versus by reference, say what is passed (a copy versus a reference or address of the original) and the consequence for the original variable (protected versus changed); one mark is often given for each half. Questions asking for advantages of local variables or disadvantages of globals expect specific points such as independent testing, reuse of identifiers and hard-to-trace changes. Benefits of modularity are best written as two-part points, giving the feature and its consequence, for example 'each module can be tested independently, so errors are easier to locate'. Avoid vague answers such as 'it is easier' or 'it is quicker' without saying why. In code questions, check that the header has the correct return type, parameter types and order, and that a function returns a value on every path. Common errors include confusing parameters with arguments, forgetting that a function call must be used in an expression, and claiming that subroutines make programs run faster.

? Check yourself

1. State the difference between a procedure and a function. [1]
2. Explain the difference between a parameter and an argument, using an example. [3]
3. Write the header of a C# function called CalculateFare that takes the number of zones (an integer) and whether the passenger is a student (Boolean) and returns the fare as a real number. [3]

Answers: 1. A function returns a value (to the calling code) whereas a procedure does not (1) 2. A parameter is a variable in the subroutine header that receives a value (1) / an argument is the actual value/variable passed in the call (1) / valid example identifying both, e.g. in AddVat(double price) price is the parameter and in AddVat(40) 40 is the argument (1) 3. static double CalculateFare(int zones, bool isStudent) – return type double (1) / name and int parameter (1) / bool parameter (1). Accept decimal/float for double; any sensible parameter names.